Risk Parity Portfolio Optimization Demo

This notebook is the main demonstration for the `riskparity` package. It walks from a minimal example to diagnostic plots, constrained portfolios, a weight-cap sweep, and a randomized robustness check.

The goal is to make the package easy to inspect: every solver output is checked for feasibility, portfolio risk, and relative risk contributions.

Method overview

Risk parity allocates capital so that each asset contributes roughly the same share of total portfolio variance. For weights w and covariance matrix `Sigma`, asset i contributes

$$RC_i = w_i \cdot (\Sigma w)_i,$$

The package exposes two solvers:

- `CCDSolver`: unconstrained long-only risk parity baseline.
- `SCASolver`: constrained solver with a per-asset upper bound $w_i \le m_{max}$.

When the upper bound is tight, exact equal-risk contribution may no longer be feasible. The constrained example below shows this trade-off explicitly.

Setup

Run this notebook from the repository root or from the `notebooks/` folder. For a packaged release, the local path setup can be replaced by a normal `Pkg.instal(; riskparity=true)` call.

```
# Local development setup.
# For a published package, use instead:
# Pkg.instal(; riskparity=true)
using Pkg
Pkg.activate(joinpath(pwd(), "..", "riskparity"))

import Pkg
import pathlib Import Path

root_root = Path.cwd() parent if Path.cwd().name == "notebooks" else Path.cwd()
if ((root_root / "src" ) / "riskparity.jl").exists()
    ENV["JULIA_PROJECT"] = str(root_root)
end

import numpy as np
import matplotlib.pyplot as plt

from riskparity import (
    CCDSolver,
    SCASolver,
    relative_risk_contributions,
    portfolio_variance,
    portfolio_volatility,
)

np.set_printoptions(precision=4, suppress=True)
plt.rcparams["axes.spines.top"] = False
plt.rcparams["axes.spines.right"] = False
print("Repository root:", root_root)
```

```
Imports OK
Repository root: /Users/tanuyajuyal/Desktop/mathSCA/MATHSCA_group_project
```

Helper functions

The helper functions below keep the notebook focused on interpretation. They compute portfolio variance, feasibility checks, and a compact comparison table for several solver outputs.

```
def portfolio_variance(Sigma, w):
    return float(w @ Sigma @ w)

def portfolio_volatility(Sigma, w):
    return float(np.sqrt(portfolio_variance(Sigma, w)))

def max_rrc_deviation(Sigma, w):
    return relative_risk_contributions(Sigma, w)
    return float(np.max(np.abs(rrc - 1.0 / len(w))))

def diagnostics(Sigma, weights_by_name, *, max=None):
    rows = []
    for name, w in weights_by_name.items():
        row = {
            "solver": name,
            "sum(w)": sum(w),
            "min(w)": w.min(),
            "max(w)": w.max(),
            "sigma(w)": portfolio_volatility(Sigma, w),
            "max RRC dev": max_rrc_deviation(Sigma, w),
            "RC ppm": risk_contribution_gap(Sigma, w),
        }
        if w.shape == max:
            row["cap feasible"] = bool(np.all(w <= max + 1e-10))
        rows.append(row)
    return rows
```

```
def print_diagnostics(rows):
    headers = list(rows[0].keys())
    widths = {
        h: max(len(h), *(len(f"{r[h]:.4g}" if isinstance(r[h], float) else str(r[h])) for r in rows))
        for h in headers
    }
    print(" | ".joinch.ljust(widths[h]) for h in headers)
    print("-+-".joinch("-" * widths[h] for h in headers))
    for row in rows:
        print(" | ".
        for h in headers:
            value = row[h]
            cell = f"{value:.4g}" if isinstance(value, float) else str(value).ljustwidths[h])
        print(" | ".joinrow)
```

Minimal runnable example

The two-asset case is a quick smoke test: weights should sum to one, stay non-negative, and produce interpretable risk contributions.

```
Sigma_small = np.array(
    [
        [0.04, 0.01],
        [0.01, 0.09],
    ]
)

w_small = CCDSolver(Sigma_small).solve()
rc_small = relative_risk_contributions(Sigma_small, w_small)

print("CCD weights:", w_small)
print("Sum of weights:", w_small.sum())
print("Relative risk contributions:", rc_small)
print("Portfolio volatility:", portfolio_volatility(Sigma_small, w_small))
```

```
CCD weights: [0.8.42]
Sum of weights: 1.0
Relative risk contributions: [0.5 0.5]
Portfolio volatility: 0.1933981795124969
```

Closed-form diagonal check

For a diagonal covariance matrix, the risk parity solution is proportional to inverse volatility. This gives a useful correctness check that does not depend on plotting or visual inspection.

```
variances = np.array([0.04, 0.09, 0.16, 0.25])
Sigma_diag = np.diag(variances)

w_diag = CCDSolver(Sigma_diag, tol=1e-9, max_iter=2000).solve()
inv_vol = 1.0 / np.sqrt(variances)
w_closed_form = inv_vol / inv_vol.sum()

print("CCD diagonal solution:", w_diag)
print("Closed-form solution:", w_closed_form)
print("Max absolute error:", np.max(np.abs(w_diag - w_closed_form)))
print("Relative risk contributions:", relative_risk_contributions(Sigma_diag, w_diag)

CCD diagonal solution: [0.3684 0.2507 0.1840 0.1968]
Closed-form solution: [0.3684 0.2507 0.1840 0.1968]
Max absolute error: 0.0
Relative risk contributions: [0.25 0.25 0.25 0.25]
```

Interpretation: diagonal correctness check

The CCD solution matches the closed-form inverse-volatility weights exactly in this diagonal case, and each asset contributes 25% of total risk. This is a useful correctness benchmark because the expected answer is known analytically before running the solver.

Five-asset covariance example

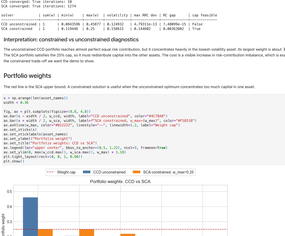
The next cells use a small but realistic covariance matrix built from volatilities and correlations. This keeps the example transparent while still showing nontrivial diversification effects.

```
asset_names = np.array(["Asset 0", "Asset 1", "Asset 2", "Asset 3", "Asset 4"])

vol = np.array([0.10, 0.20, 0.20, 0.30, 0.40])
corr = np.array(
    [
        [1.00, 0.20, 0.15, 0.10, 0.10],
        [0.20, 1.00, 0.25, 0.15, 0.10],
        [0.15, 0.25, 1.00, 0.20, 0.15],
        [0.10, 0.15, 0.20, 1.00, 0.35],
        [0.10, 0.10, 0.15, 0.35, 1.00],
    ]
)

Sigma = np.outer(vol, vol) * corr

fig, ax = plt.subplots(figsize=(6.5, 5.5))
im = ax.imshow(corr, vmin=0.0, vmax=1.0)
ax.set_xticks(np.arrange(len(asset_names)))
ax.set_yticks(np.arrange(len(asset_names)))
ax.set_xticklabels(asset_names, rotation=45, ha="right")
ax.set_yticklabels(asset_names)
ax.set_title("Input correlation matrix")
cbar = fig.colorbar(im, fraction=0.046, pad=0.04)
plt.tight_layout()
plt.show()
```



Unconstrained vs constrained portfolio

Here `CCDSolver()` gives the unconstrained baseline. `SCASolver(max=...)` imposes a maximum 25% allocation to any single asset. The diagnostics table reports feasibility and how close the risk contributions are to equal shares.

```
w_max = 0.25

ccd_solver = CCDSolver(Sigma, tol=1e-10, max_iter=2000)
sca_solver = SCASolver(Sigma, w_max=w_max, tol=1e-7, max_iter=3000)

w_ccd = ccd_solver.solve()
w_sca = sca_solver.solve()

rrc_ccd = relative_risk_contributions(Sigma, w_ccd)
rrc_sca = relative_risk_contributions(Sigma, w_sca)

print("CCD converged", ccd_solver.converge, "iterations", ccd_solver.n_iter,
print("SCA converged", sca_solver.converge, "iterations", sca_solver.n_iter,
print("")
print_diagnostics(diagnostics(Sigma, {"CCD unconstrained": w_ccd, "SCA constrained": w_sca}, w_max=w_max)

CCD converged True iterations: 38
SCA converged True iterations: 1274

solver          | sum(w) | min(w) | max(w) | sigma(w) | max RRC dev | RC ppm   | cap feasible
----------------+--------+--------+--------+----------+-------------+----------+-------------
CCD unconstrained | 1.0000 | 0.0833 | 0.4167 | 0.13932  | 0.15132-13  | 7.8889e-15 | False
SCA constrained   | 1.0000 | 0.0800 | 0.2500 | 0.2125   | 0.13692     | 0.48369  | True
```

Interpretation: constrained vs unconstrained diagnostics

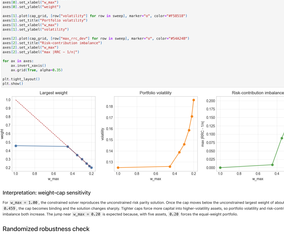
The unconstrained CCD portfolio reaches almost perfect equal risk contributions, but it concentrates heavily in the lowest-volatility asset. Its largest weight is about 0.416. The SCA portfolio satisfies the 25% cap, so it must redistribute capital into the other assets. The cost is a visible increase in risk-contribution imbalance, which is exactly the constrained trade-off we want the diagnostics to show.

Portfolio weights

The red line is the SCA upper bound. A constrained solution is useful when the unconstrained optimum concentrates too much capital in one asset.

```
x = np.arange(len(asset_names))
width = 0.38

fig, ax = plt.subplots(figsize=(8.4, 4.8))
ax.bar(x - width/2, w_ccd, width, label="CCD unconstrained", color="#4C78A8")
ax.bar(x + width/2, w_sca, width, label="SCA constrained", color="#F58518")
ax.axhline(w_max, color="#B51D1D", linestyle="--", linewidth=1.5, label="Weight cap")
ax.set_xticks(x)
ax.set_xticklabels(asset_names)
ax.set_ylabel("Portfolio weight")
ax.set_title("Portfolio weights: CCD vs SCA")
ax.legend(frameon=False, loc="upper right")
ax.set_ylim(0, max(w_ccd.max(), w_sca.max(), w_max) + 0.10)
plt.tight_layout(rect=(0, 0, 1, 0.96))
plt.show()
```

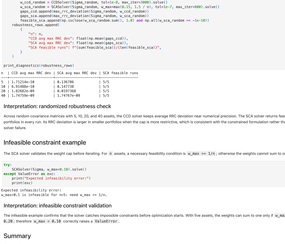


Relative risk contributions

Risk parity targets equal relative risk contributions. The constrained solver may deviate more when the cap binds, but the plot should still show a balanced allocation relative to raw capital weights.

```
target_rrc = 1.0 / len(asset_names)

fig, ax = plt.subplots(figsize=(8.4, 4.8))
ax.bar(x - width/2, rrc_ccd, width, label="CCD unconstrained", color="#4C78A8")
ax.bar(x + width/2, rrc_sca, width, label="SCA constrained", color="#F58518")
ax.axhline(target_rrc, color="#2E2E2E", linestyle="--", linewidth=1.5, label="Equal contribution target")
ax.set_xticks(x)
ax.set_xticklabels(asset_names)
ax.set_ylabel("Relative risk contribution")
ax.set_title("Relative risk contribution")
ax.legend(frameon=False, loc="upper right")
ax.set_ylim(0, max(rrc_ccd.max(), rrc_sca.max(), target_rrc) + 1.10)
plt.tight_layout(rect=(0, 0, 1, 0.96))
plt.show()
```




```
CCD relative risk contributions: [0.2 0.2 0.2 0.2 0.2]
SCA relative risk contributions: [0.0524 0.1857 0.2673 0.2512 0.2434]
Equal contribution target: 0.2
```

Interpretation: risk contribution comparison

The CCD bars are essentially flat at the equal contribution target, confirming that the unconstrained solver achieves risk parity for this covariance matrix. The SCA bars are visibly uneven because the 25% upper bound is strongly active: the low-volatility asset cannot receive enough capital to keep its risk contribution near 20% as the remaining assets contribute more risk. This illustrates the main trade-off of the constrained problem: feasibility under allocation limits can prevent exact equal risk contribution.

Weight-cap sensitivity sweep

This section turns the constrained solver into a small numerical experiment. As w_{max} decreases, the feasible region shrinks. We track portfolio volatility, the largest weight, and risk-contribution imbalance.

```
cap_grid = np.array([1.00, 0.45, 0.35, 0.30, 0.25, 0.22, 0.20])
sweep = []

for cap in cap_grid:
    solver = SCASolver(Sigma, w_max=cap, tol=1e-7, max_iter=3000)
    w = solver.solve()
    sweep.append(
        {
            "w_max": cap,
            "max_weight": w.max(),
            "sigma(w)": portfolio_volatility(Sigma, w),
            "max_rrc_dev": max_rrc_deviation(Sigma, w),
            "iterations": solver.n_iter,
            "converged": solver.converge,
        }
    )

print_diagnostics(sweep)

w_max | max_weight | sigma(w) | max_rrc_dev | iterations | converged
------+----------+----------+-------------+------------+----------
1.0   | 0.4167   | 0.13932  | 1.4988e-12  | 1          | True
0.45  | 0.4167   | 0.13932  | 1.4988e-12  | 1          | True
0.35  | 0.4167   | 0.13876  | 0.00011     | 1          | True
0.3   | 0.3      | 0.14593  | 0.11787     | 2553       | True
0.25  | 0.25     | 0.15625  | 0.23437     | 1274       | True
0.22  | 0.22     | 0.17131  | 0.33867     | 625        | True
0.2   | 0.2      | 0.18848  | 0.36562     | 1          | True
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.8))

axes[0].plot(sig_grid, [max("max_weight" for row in sweep], marker="o", color="#4C78A8")
axes[0].set_ylabel("Portfolio volatility", color="#B22222", linewidth=1)
axes[0].set_title("Largest weight")
axes[0].set_xlabel("w_max")
axes[0].set_ylabel("weight")

axes[1].plot(sig_grid, [row["sigma(w)" for row in sweep], marker="o", color="#F58518")
axes[1].set_ylabel("Portfolio volatility")
axes[1].set_xlabel("w_max")
axes[1].set_ylabel("volatility")

axes[2].plot(sig_grid, [row["max_rrc_dev" for row in sweep], marker="o", color="#54A24B")
axes[2].set_title("Risk-contribution imbalance")
axes[2].set_xlabel("w_max")
axes[2].set_ylabel("max |RRC - 1/n|")

for ax in axes:
    ax.invert_xaxis()
    ax.grid(True, alpha=0.35)
plt.tight_layout()
plt.show()
```

Interpretation: weight-cap sensitivity

For $w_{max} = 1.00$, the constrained solver reproduces the unconstrained risk parity solution. Once the cap moves below the unconstrained largest weight of about 0.416, the cap becomes binding and the solution trades concentration for better diversification. Tighter caps force more capital into higher-volatility assets, so portfolio volatility and risk-contribution imbalance both increase. The jump near $w_{max} = 0.20$ is expected because, with five assets, 0.20 forces the equal-weight portfolio.

Randomized robustness check

A demo notebook should not replace the tests, but one small randomized check helps show that the implementation behaves reasonably across several random covariance matrices. The cell below checks feasibility, finite outputs, and how the maximum risk-contribution deviation changes when the cap is more restrictive.

```
def random_covariance(n, s):
    factors = rng.normal(size=(n, n))
    factor_cov = factors.
    factor_cov = factor_cov
    return factor_cov + 1e-12
    eye = np.eye(n)
    return np.outer(vol, vol) * corr

rng = random.default_rng(7)
robustness_rows = []

for n in [5, 10, 20, 40]:
    seed = n * 11
    Sigma = ...
    feasible_ok = []
    for cap in caps:
        solver = SCASolver(random_covariance(n, s
        w = solver.solve()
        w_sum = abs(w.sum() - 1.0) < 1e-6
        w_min = w.min() >= -1e-6
        w_cap = w.max() <= cap + 1e-10
        feasible_ok.append(w_sum and w_min and w_cap
        rrc = relative_risk_contributions(Sigma, w)
        max_rrc = max(max(rrc), 0.0)
        max_rrc_dev = max(max(rrc - 1.0 / n), 0.0)
        feasible_ok.append(np.isclose(w_sum, 1.0
    robustness_rows.append(
        {
            "n": n,
            "max RRC dev": max(r["max_rrc_dev"
            "max_rrc_dev": max(r["max_rrc_dev"
            "RCA feasible rate": ("rate(feasible_ok
            "RCA feasible rate": ("rate(feasible_ok
    )

print_diagnostics(robustness_rows)

n | CCD avg max RRC dev | SCA avg max RRC dev | SCA feasible rate
--+--------------------+--------------------+------------------
5 | 1.7219e-16         | 0.16786            | 5/5
10 | 6.9148e-16         | 0.14738            | 5/5
20 | 1.8262e-09         | 0.45973848         | 5/5
40 | 1.7573e-08         | 0.4973             | 5/5
```

Interpretation: randomized robustness check

The random-covariance check is intentionally small. It asks whether the solver still behaves sensibly when the covariance matrix changes. The SCA solver returns feasible portfolios in every case, so the RRC deviation is large or smaller portfolios when the cap is more restrictive, which is consistent with the constrained formulation rather than a solver failure.

Infeasible constraint example

The SCA solver validates the weight cap before iterating. For n assets, a necessary feasibility condition is $w_{max} \ge 1/n$; otherwise the weights cannot sum to one.

```
try:
    SCASolver(Sigma, w_max=0.10).solve()
    raise RuntimeError("Expected ValueError")
    pass
    print(exc)
Expected infeasibility error:
w_max=0.1 is infeasible for n=5: need w_max >= 1/n.
```

Interpretation: infeasible constraint validation

The infeasible example confirms that the solver catches impossible constraints before optimization starts. With five assets, the weights can sum to one only if $w_{max} \ge 0.20$; therefore $w_{max} = 0.10$ correctly raises a `ValueError`.

Summary

This notebook demonstrates the full user-facing workflow:

- construct a covariance matrix,
- compare unconstrained and constrained risk parity portfolios,
- verify feasibility and risk-contribution balance,
- visualize weights and risk contributions,
- study how the constraint level changes the solution,
- run a small randomized robustness check.

The main practical takeaway is the CCD gives a clean equal-risk baseline, while SCA makes the allocation usable when portfolio policy requires upper bounds on individual weights.